II. Principii S.O.L.I.D.

1. Care este contractul următoarei metode?

```
public static Collection<String> readLinesFromFile(String absoluteFilePath)
```

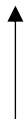
Precondiții:
Postcondiții:
Invarianți:
Efecte secundare:

2. Ce principiu este încălcat în următorul exemplu? Explicați răspunsul. Cum ați refactoriza codul pentru a nu încălca principiul?

```
interface IMembership {
   void login(UserCredentials uc);
   boolean logout();
   UserCredentials register(String userName, String password);
   UserCredentials changePassword(String userName);
   void addPaymentMethod(UserCredentials uc, PaymentMethod pm);
}
```

3. Respectă următoarea metodă principiul „design by contract”? De ce? Cum puteți asigura că contractul metodei de bază nu va fi încălcat niciodată de o metodă care o suprascrie?

```
String Base :: aMethod(int[] args)
//PRECONDITIONS: args contains only negative numbers
//POSTCONDITIONS: returns a String with only lowercase characters or digits
```



```
String Derived :: aMethod(int[] args)
//PRECONDITIONS: args contains any negative or even numbers
//POSTCONDITIONS: returns a String composed only of digits or null
```

4. Imaginați-vă că aveți o clasă ProductsOffer care stochează o colecție de Produse. Fiecare produs are un nume, un nume al producătorului, o culoare, un preț, o greutate și o dată de lansare. În prima versiune, ProductsOffer oferă următoarele servicii pentru filtrarea produselor:

```
getByName()
getByManufacturerName()
getByColor()
getByPrice()
getByWeight()
getByReleaseDate()
```

După un timp, clientul are nevoie să filtreze după nume și culoare, așa că adăugăm metoda: getByNameAndColor()

După un alt timp, clientul are nevoie să filtreze după greutate și dată, așa că adăugăm metoda: getByWeightAndReleaseDate()

Clasa este bine proiectată? Ce principiu este încălcat? Explicați de ce. Cum ar trebui să refactorizați clasa astfel încât să se poată adapta ușor la nevoile viitoare ale clientului? Implementați codul.

5. Ce principiu este încălcat în următorul exemplu? Explicați de ce. Cum ați refactoriza codul pentru a nu încălca principiul?

```
abstract class Vehicle {
   private int peopleCapacity;

   abstract double getSpeed();
   abstract double getGasLevel();
   abstract void pressClutch();
 }

  class Scooter extends Vehicle{
        //does not have a clutch
 }
```

```
class Tesla extends Vehicle{
      //Tesla is an Electric Car
      int getTrunkCapacity(){...}
      int getFrunkCapacity(){...}
      double getAutonomy(){...}
}

class Logan extends Vehicle{
      int getTrunkCapacity(){...}
}
```

6. Ce principiu este încălcat în următorul exemplu? Explicați de ce. Cum ați refactoriza codul? Desenați o diagramă de clase care să arate soluția.

```
class Student {
  …
  public void saveStudent();
  public Double calculateMeanGrade();
  public Profile getStudentProfile();
}
```